SEMANA 13 · DOS/DDOS

Denegación de servicio: cómo se cae un servicio sin entrar en él

Parte 1 de 4 · Unidad 3

Ing. Rodolfo Cañas Cervantes
Universidad de la Costa (CUC) · Ingeniería de Sistemas
Semana 13 · Periodo 2026-2 · Barranquilla, Colombia

Qué es un ataque de denegación de servicio, por qué funciona y cómo se detecta antes de mitigarlo.

El objetivo es la disponibilidad

En la tríada CIA, este ataque golpea la D: no roba datos, apaga el servicio.

Las tres propiedades y a cuál golpea este ataque



CONFIDENCIALIDAD

que no lo vea

quien no debe



INTEGRIDAD

que no se altere

lo que existe



DISPONIBILIDAD

que esté cuando

se necesita

El DoS/DDoS no roba ni altera datos: apaga el servicio.

Fuente: elaboración propia; NIST SP 800-61 Rev. 3 (2025)

Fuente: elaboración propia; NIST SP 800-61 Rev. 3 (2025)

DoS y DDoS: la diferencia está en el origen

Una sola fuente satura; miles de fuentes distribuidas saturan y además esconden al autor.

Una fuente contra miles de fuentes



DoS

una sola fuente

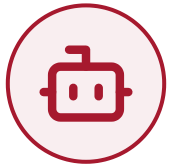
bloquear la IP sirve



DDoS

miles de fuentes

distribuidas



BOTNET

equipos infectados

obedecen órdenes

Contra miles de fuentes, bloquear IPs bloquea también a tus usuarios.

Fuente: elaboración propia; CISA — guía de DDoS (2026)

Fuente: elaboración propia; CISA — guía de DDoS (consultada 2026)

Tres familias de ataques

Volumen, protocolo y aplicación: cada familia golpea una capa distinta y se detecta distinto.

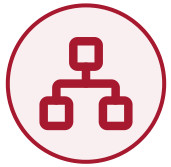
Tres familias y su unidad de medida



VOLUMEN

satura el enlace

Gbps



PROTOCOLO

agota una tabla

o un estado · pps



APLICACIÓN

consume el servicio

rps

Cada familia se detecta con una métrica distinta y se mitiga en otro punto.

Fuente: Cloudflare Learning — tipos de DDoS (consultado 2026)

Fuente: elaboración propia a partir de Cloudflare Learning — tipos de DDoS (consultado 2026)

El three-way handshake

SYN, SYN-ACK, ACK: tres pasos que crean estado en el servidor y que un atacante puede abusar.

El saludo de tres vías crea estado



1 · SYN

el cliente pide

abrir conexión



2 · SYN-ACK

el servidor reserva

estado y responde



3 · ACK

la conexión queda

establecida

Ese estado reservado es finito: ahí está la vulnerabilidad.

Fuente: elaboración propia; RFC 9293 (TCP, 2022)

Fuente: elaboración propia; RFC 9293 (TCP, 2022)

SYN flood: agotar la tabla de conexiones

Muchos SYN sin ACK llenan la cola del servidor; las conexiones legítimas ya no caben.

SYN flood: llenar la cola de conexiones

MILES DE SYN

origen falsificado

nadie responde

SYN-ACK SIN ACK

el servidor reserva

y espera en vano

COLA LLENA

las conexiones reales

ya no caben

Firma en las métricas: muchos SYN de entrada y pocos ACK.

Fuente: RFC 9293 (2022); CISA — guía de DDoS (2026)

Fuente: elaboración propia; RFC 9293 (2022); CISA — guía de DDoS (2026)

HTTP flood: agotar el servidor web

Peticiones que parecen normales, pero muchas y muy seguidas: el servidor trabaja hasta caer.

HTTP flood: peticiones válidas en exceso



PETICIONES

HTTP completas

y válidas



VOLUMEN ALTO

base de datos

e intérprete saturados



SERVICIO CAÍDO

los usuarios reales

no reciben respuesta

Firma en las métricas: rps se dispara y suben los tiempos de respuesta.

Fuente: CISA — guía de DDoS (2026); nginx docs (2026)

Fuente: elaboración propia; CISA — guía de DDoS (2026); nginx docs — limit_req (2026)

Botnets y amplificación

Miles de equipos infectados más servidores que amplifican: poco esfuerzo, mucho daño.

Botnet más amplificación



CONSULTA PEQUEÑA

origen falsificado:

la víctima

SERVIDOR ABIERTO

responde mucho

más de lo recibido

VÍCTIMA

recibe una respuesta

enorme que no pidió

Con pocos servidores mal configurados se multiplica el tráfico.

Fuente: CISA (2026); Cloudflare Learning (2026)

Fuente: elaboración propia; CISA — guía de DDoS (2026); Cloudflare Learning (2026)

Las tres métricas que hay que mirar

pps, bps y rps: cada familia de ataque deja su firma en una de las tres.

Tres métricas, tres firmas



pps

paquetes por segundo

ataques de protocolo



bps

bits por segundo

ataques de volumen



rps

peticiones por segundo

ataques de aplicación

Ninguna métrica sola detecta todo: hay que mirar las tres.

Fuente: nftables wiki (2026); nginx docs (2026)

Fuente: elaboración propia; nftables wiki — limit rate (2026); nginx docs — limit_req (2026)

Línea base y umbrales

Un umbral solo sirve si conoces tu tráfico normal; sin línea base, todo parece ataque.

El umbral vive sobre una línea base



LÍNEA BASE

qué es normal

en tu red



PICO PREVISIBLE

horarios y

respaldos



PICO ANÓMALO

lo que se sale

del patrón

El umbral no es universal: es un número de tu red.

Fuente: elaboración propia; Suricata docs — threshold (2026)

Fuente: elaboración propia; Suricata docs — threshold (2026)

Firmas contra anomalías

Una firma reconoce lo que ya viste; una anomalía marca lo que se sale del patrón.

Firmas contra anomalías



FIRMA

reconoce lo visto

precisa y explicable



ANOMALÍA

marca lo que se

sale del patrón



COMPLEMENTO

ambas cubren

lo que la otra no ve

En este laboratorio usamos firmas porque se verifican de forma determinista.

Fuente: Suricata docs (2026); material de clase S12

Fuente: elaboración propia; Suricata docs (2026); material de clase S12

Por qué el umbral fijo se queda corto

Cuando el tráfico legítimo cambia, el umbral escrito a mano envejece y hay que reajustarlo.

Los tres límites de la regla fija



A MANO

una regla por cada ataque conocido



SE DESACTUALIZA

si el tráfico legítimo cambia



NO VE VARIANTES

cambios sutiles de un ataque conocido

No es una crítica: es la razón por la que la unidad avanza hacia modelos.

Fuente: elaboración propia a partir de Gartner — NDR (2026)

Fuente: elaboración propia a partir de Gartner — NDR (2026)

Qué ha cambiado en los últimos años

Ataques más grandes, más baratos y con motivación económica: el contexto de las amenazas actuales.

Qué cambió en el contexto



MÁS VOLUMEN

ataques más grandes

que antes



MÁS BARATOS

menos esfuerzo

para el atacante



EXTORSIÓN

atacar y luego vender

el antídoto

La disponibilidad se diseña en la arquitectura, no se parcha después.

Fuente: ENISA Threat Landscape 2024; CISA (2026)

CASO

Caso ilustrativo: la tarde en que no se pudo cobrar

Caso de clase, no es una empresa real: una tienda en línea recibe un flood y pierde ventas sin saber por qué.

La tarde en que no se pudo cobrar



USUARIOS

la página tarda

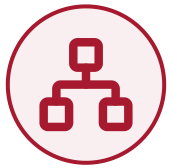
y deja de responder



SERVIDOR

procesador y base

de datos sin errores



RED

pico de peticiones

desde muchos orígenes

Si solo miras la capa de aplicación, un ataque de red te parece un misterio.

Caso ilustrativo de clase: no es una empresa real y va sin cifras.

Fuente: caso ilustrativo de clase; sin datos de una empresa real y sin cifras

SÍNTESIS

Lo que queda de la Parte 1

Tres ideas: el objetivo es la disponibilidad, tres familias con tres métricas, y el umbral necesita línea base.

Síntesis de la Parte 1



1 · DISPONIBILIDAD

el objetivo del

ataque



2 · TRES FAMILIAS

volumen, protocolo

y aplicación



3 · LÍNEA BASE

sin observar no hay

umbral válido

Con esto listo, la Parte 2 entra al laboratorio.

Fuente: elaboración propia; NIST SP 800-61 Rev. 3 (2025)

Fuente: elaboración propia; NIST SP 800-61 Rev. 3 (2025)

SEMANA 13 · PARTE 2

Laboratorio 6: medir, capturar y entender el ataque

Parte 2 de 4 · Unidad 3

Ing. Rodolfo Cañas Cervantes

Universidad de la Costa (CUC) · Ingeniería de Sistemas

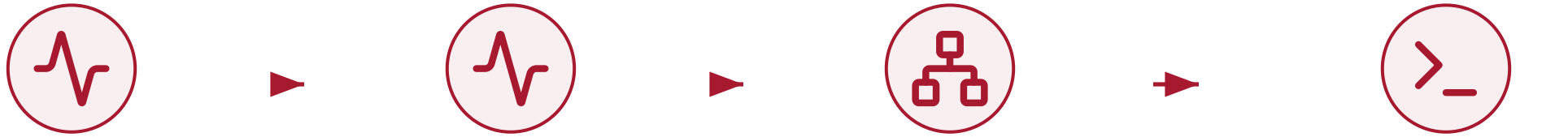
Semana 13 · Periodo 2026-2 · Barranquilla, Colombia

Cómo se arranca el Lab 6, cómo se mide la línea base y cómo se leen las capturas con tcpdump.

Lab 6 en una frase

Detectar con Suricata y mitigar con nftables un SYN flood, un HTTP flood y un ataque multi-origen.

Tres ataques, dos herramientas, un servidor



SYN flood

protocolo

tabla de conexiones

HTTP flood

aplicación

peticiones válidas

multi-origen

distribuido

muchas fuentes

Suricata + nftables

detectar y mitigar

en el contenedor fw

Entorno cerrado: sin puertos publicados, sin internet en ejecución, solo redes internas.

Fuente: Lab 6 v1.0.4 (guía del laboratorio) · formativa · ~90 min

TOPOLOGÍA

Direccionamiento y contenedores

Dos redes internas: clientes en 10.20.10.0/24 y servidor en 10.30.0.0/24, con el web en 10.30.0.10.

Dos redes internas con el firewall en el medio

LAN CLIENTES · 10.20.10.0/24

cliente legítimo

10.20.10.21

generador de tráfico

10.20.10.50

gateway 10.20.10.1 · fw 10.20.10.2



LAN SERVIDOR · 10.30.0.0/24

servidor web objetivo

10.30.0.10

gateway 10.30.0.1

fw 10.30.0.2

El objetivo es siempre 10.30.0.10: el generador solo reproduce capturas pregrabadas.

Fuente: Lab 6 v1.0.4 · docker-compose del paquete

Antes de empezar: instalar y arrancar

El cliente asur-lab se instala una vez; después basta con iniciar el lab06 y entrar a su carpeta.

Instalar una vez, arrancar y entrar

en tu VM de Ubuntu

```
curl -fsSL https://asur-col.github.io/cuc-seguridad-redes/laboratorios/instalar-asur-lab.sh | bash
export PATH="$HOME/.local/bin:$PATH"
asur-lab iniciar lab06
cd ~/labs/lab06
```

Si ya lo tenías instalado: vuelve a ejecutar asur-lab iniciar lab06 y se actualiza sin borrar tus archivos.

Fuente: Lab 6 v1.0.4 · sección 0, Antes de empezar

Los diez comandos del laboratorio

up, down, reset, estado, base, ataque, recargar, check, shell y evidencia: todo el control en ./lab.

Todo el control del laboratorio en un solo script



`./lab up`

arranca



`./lab down`

apaga



`./lab reset`

reinicia



`./lab estado`

diagnóstico



`./lab base`

línea base



`./lab ataque`

syn | http | multi



`./lab recargar`

aplica reglas



`./lab check`

6 objetivos + bono



`./lab shell`

entra a un contenedor



`./lab evidencia`

genera el tar.gz

Casi nunca tocas Docker directamente: todo pasa por ./lab.

Fuente: Lab 6 v1.0.4 · comandos disponibles

Primero se mide: ./lab base 30

Treinta segundos sin atacar para saber qué es normal: pps, conexiones SYN-RECV y orígenes.

Primero se mide, después se escribe la regla

```
medición de 30 s sin atacar
$ ./lab base 30
[INFO] midiendo la línea base durante 30 s (no lances ataques)
[OK] escrito ~/labs/lab06/evidencia/linea-base.txt
pps=22 · SYN-RECV=0 · orígenes=3
```



pps 3 a 60

paquetes por segundo



SYN-RECV 0 a 50

conexiones a medio abrir



orígenes 1 a 5

direcciones distintas

Son valores para comparar, no para memorizar: tu línea base es tu referencia.

Fuente: Lab 6 v1.0.4 · sección 1, Línea base (15 min)

EVIDENCIA

Qué guarda linea-base.txt

El archivo registra pps, SYN-RECV y orígenes únicos: es el objetivo 1 de ./lab check.

Tres números que describen tu tráfico normal



pps

ritmo normal

de paquetes



SYN-RECV

conexiones a medio

abrir esperando ACK



orígenes

cuántas IP

están hablando

Es el objetivo 1 de ./lab check: sin este archivo no hay evidencia de haber medido.

Fuente: Lab 6 v1.0.4 · campos de linea-base.txt

Análisis de capturas: los pcap de /datos

Cuatro capturas pregrabadas dentro del contenedor fw: normal, syn-flood, http-flood y multi-origen.

Cuatro capturas pregrabadas dentro del contenedor fw



`normal.pcap`

tráfico legítimo



`syn-flood.pcap`

SYN sin ACK



`http-flood.pcap`

peticiones GET



`multi-origen.pcap`

muchas fuentes

Se repiten cuantas veces quieras: mismo resultado, comparable y sin riesgo.

Fuente: Lab 6 v1.0.4 · sección 2, Análisis de capturas (20 min)

tcpdump: leer una captura sin abrir Wireshark

Contar paquetes, ver la primera y la última marca de tiempo: el tamaño y la duración del ataque.

Tres comandos: tamaño, contenido y duración

```
dentro del contenedor: ./lab shell fw
tcpdump -nn -r /datos/syn-flood.pcap | head -3
tcpdump -nn -r /datos/syn-flood.pcap | wc -l
tcpdump -tt -nn -r /datos/syn-flood.pcap | sed -n '1p;$p'
```

Con el total de paquetes y la duración ya puedes calcular la tasa.

Fuente: Lab 6 v1.0.4 · sección 2, comandos sobre /datos/syn-flood.pcap

¿Cuántos orígenes y qué método HTTP?

Contar direcciones únicas separa un ataque de una fuente de uno distribuido; -A revela el método.

Dos tuberías que responden dos preguntas

```
contar orígenes · ver el método
tcpdump -nn -r /datos/syn-flood.pcap | awk '{print $3}' | cut -d. -f1-4 | sort -u | wc -l
tcpdump -A -nn -r /datos/http-flood.pcap | grep -a -m3 'HTTP/1'
```



una sola IP

ataque de una fuente

se puede bloquear



muchas IP

ataque distribuido

necesita límite de tasa

Fuente: Lab 6 v1.0.4 · sección 2, awk, cut y grep sobre las capturas

PLANTILLA

Los cinco campos de analisis-pcap.txt

Bandera dominante, orígenes, tasa, método HTTP y la explicación del medio saludo.

Cuatro números y una explicación escrita



`flag_dominante`

qué bandera TCP domina



`origenes_syn`

cuántas IP originan el flood



`tasa_syn_pps`

paquetes ÷ segundos



`http_flood`

qué método HTTP envía



`diferencia`

por qué satura sin completar el saludo

Se evalúa que sepas medir y que sepas explicar: solo números no alcanza.

Fuente: Lab 6 v1.0.4 · analisis-pcap.txt (objetivo 2)

Calcular la tasa sin equivocarse

Paquetes totales divididos entre la duración en segundos, y ojo con las unidades: es pps, no bps.

Tres pasos y dos errores típicos



1 · CONTAR

paquetes totales

con wc -l

2 · DURACIÓN

última marca menos

primera marca

3 · DIVIDIR

paquetes ÷ segundos

= pps

Errores típicos: confundir pps con bps y restar mal las marcas de tiempo.

Fuente: Lab 6 v1.0.4 · campo tasa_syn_pps

El marco ético del laboratorio

El generador solo reproduce capturas contra 10.30.0.10; no hay herramientas para atacar terceros.

Por qué este laboratorio no es un ataque real



DESTINO FIJO

solo 10.30.0.10

red interna Docker



NO CONFIGURABLE

tasas y duración

fijadas en el código



SIN HERRAMIENTAS

nada para atacar

sistemas de terceros

Ley 1273 de 2009: atacar un sistema ajeno es delito, aunque sea para probar.

Fuente: Lab 6 v1.0.4 · marco ético; Ley 1273 de 2009 (Colombia)

LECTURA

Cómo se lee una línea de tcpdump

Hora, origen, destino y banderas: cuatro columnas que cuentan toda la historia del paquete.

Las cuatro columnas de tcpdump



1 · HORA

marca de tiempo:

sirve para la duración



2 · ORIGEN

IP y puerto de

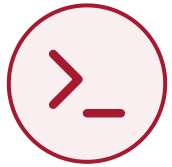
quien envía



3 · DESTINO

IP y puerto: revela

el servicio



4 · BANDERAS

entre corchetes:

saludo, respuesta o datos

Con esas cuatro columnas respondes quién, a quién, con qué y a qué ritmo.

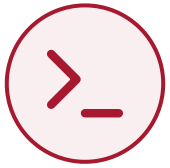
Fuente: Lab 6 v1.0.4 · salida de tcpdump sobre /datos

SÍNTESIS

Lo que queda de la Parte 2

Arrancar, medir la línea base y leer las capturas: el ataque se entiende antes de defenderse.

Lo que ya sabes hacer al terminar esta parte



ARRANCAR

asur-lab iniciar lab06

./lab estado



MEDIR

./lab base 30

pps, SYN-RECV, orígenes



LEER

tcpdump sobre /datos

cinco campos

Entendiste el ataque con datos propios: eso es la mitad de la defensa.

Fuente: Lab 6 v1.0.4 · secciones 0, 1 y 2

SEMANA 13 · PARTE 3

Detectar con umbrales y mitigar con límites de tasa

Parte 3 de 4 · Unidad 3

Ing. Rodolfo Cañas Cervantes
Universidad de la Costa (CUC) · Ingeniería de Sistemas
Semana 13 · Periodo 2026-2 · Barranquilla, Colombia

Las dos reglas de Suricata con umbral y las tres mitigaciones de nftables, sin entregar la solución resuelta.

Qué pide la sección de detección

Dos reglas con threshold en reglas/lab.rules, recargar y verificar alertas en eve.json.

Escribir, recargar, atacar y verificar



dos causas de que no haya alerta

sid repetido

umbral imposible de alcanzar

el check exige

al menos 1 alerta por regla

Fuente: Lab 6 v1.0.4 · sección 3, Detección (20 min)

La plantilla trae las reglas comentadas

Están en lab.rules con numeral al inicio: quitas el comentario y reemplazas N y M.

Comentadas, con marcadores N y M

estado inicial: todo comentado

```
# alert tcp any any -> any 80 (msg:"SYN flood"; flags:S;
# threshold: type threshold, track by_src, count N, seconds M; sid:1000001; rev:1;)
```

estado final: sin # y con N y M elegidos por ti

```
alert tcp any any -> any 80 (msg:"SYN flood"; flags:S;
threshold: type threshold, track by_src, count N, seconds M; sid:1000001; rev:1;)
```

La guía lo dice explícito: ejemplo sin resolver. Elegir el umbral es el ejercicio.

Fuente: Lab 6 v1.0.4 · plantilla reglas/lab.rules

Anatomía de una regla Suricata

Cabecera, mensaje, banderas, contenido, umbral e identificador: cada parte tiene una función.

Seis partes que se leen como una frase



`CABECERA`

de dónde a dónde



`msg`

lo que verás en la alerta



`flags`

qué bandera TCP



`content`

cadena en el paquete



`threshold`

cuántas veces antes de alertar



`sid / rev`

identificador único

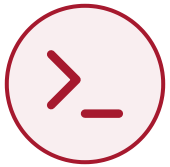
Fuente: Suricata docs · formato de cabecera y opciones (2026)

Fuente: Suricata docs — reglas y formato de cabecera (consultado 2026)

threshold: contar por origen y en el tiempo

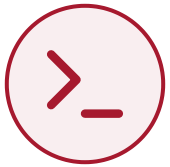
type threshold, track by_src, count N, seconds M: la alerta se dispara al superar N en M segundos.

Cuatro partes que hay que entender



`type threshold`

alerta al superar la cuenta



`track by_src`

agrupa por IP de origen



`count N`

cuántas coincidencias se permiten



`seconds M`

la ventana de tiempo

Si la misma IP supera N coincidencias en M segundos, la regla alerta.

Fuente: Suricata docs · threshold (2026); Lab 6 v1.0.4

Cómo elegir N y M sin adivinar

Compara con tu línea base: N en M segundos debe estar por encima del tráfico legítimo de una sola IP.

Medir, comparar y dejar margen



LÍNEA BASE

lo normal de una IP

medido antes

UMBRAL

entre lo normal

y el ataque

ATAQUE

muy por encima

del umbral

Ventana M: ni tan corta que un pico normal dispare, ni tan larga que el ataque termine antes.

Fuente: elaboración propia a partir de Lab 6 v1.0.4 · secciones 1 y 3

La regla del SYN flood

Banderas solo SYN hacia el puerto 80, con umbral por origen: la plantilla ya trae la estructura.

Estructura de la plantilla, con N y M por decidir

```
flags:S · solo el saludo
alert tcp any any -> any 80 (msg:"SYN flood"; flags:S;
threshold: type threshold, track by_src, count N, seconds M; sid:1000001; rev:1;)
```



por qué solo S

el ataque son saludos

que nunca se completan



por qué puerto 80

es donde escucha

el servidor web

Fuente: Lab 6 v1.0.4 · sección 3, plantilla sin resolver

La regla del HTTP flood

Banderas push y acknowledge, contenido GET al inicio del paquete y umbral por origen.

Banderas de datos más contenido GET

```
flags:PA · viajan datos
alert tcp any any -> any 80 (msg:"HTTP flood"; flags:PA; content:"GET "; depth:4;
threshold: type threshold, track by_src, count N, seconds M; sid:1000002; rev:1;)
```

flags:PA

push y acknowledge:
la conexión ya existe

content GET

una petición normal...
repetida sin fin

Fuente: Lab 6 v1.0.4 · sección 3, plantilla sin resolver

VERIFICACIÓN

Recargar y ver las alertas

./lab recargar aplica las reglas; luego el ataque y la lectura de /var/log/suricata/eve.json.

El ciclo de verificación de la detección

tres pasos, en este orden

```
./lab recargar
./lab ataque syn # y luego: ./lab ataque http
./lab shell ids -> tail /var/log/suricata/eve.json
```

si no recargas

Suricata sigue con la configuración anterior

si repites el sid

la regla se ignora y nunca verás la alerta

Fuente: Lab 6 v1.0.4 · sección 3 (recargar y eve.json)

Leer una alerta en eve.json

Cada línea es un evento en JSON: marca de tiempo, origen, destino, firma y severidad.

Cada línea es un evento independiente en JSON



`timestamp`

cuándo ocurrió



`src_ip / src_port`

quién lo envió



`dest_ip / dest_port`

hacia dónde iba



`alert.signature`

el msg de tu regla



`alert.signature_id`

cuál de tus dos reglas

Es el mismo archivo que en un SOC (centro de operaciones de seguridad) real se envía al sistema de correlación.

Fuente: Suricata docs · formato eve.json (2026)

Mitigar en nftables: limit rate over

Un límite de tasa SYN por IP con drop; sin la palabra over, descartarías al cliente legítimo.

limit rate over N/second sobre tcp flags syn

CORRECTO

```
limit rate over N/second
tcp flags syn drop
```

INCORRECTO

```
limit rate N/second
tcp flags syn drop
```

con over

coincide con lo que supera el límite

sin over

coincide con lo que va por debajo: descartas al cliente

La guía lo advierte con todas las letras: sin over descartarías al usuario legítimo.

Fuente: Lab 6 v1.0.4 · sección 4, límite de tasa SYN por IP

Limitar conexiones simultáneas

ct count over N por IP de origen protege contra muchas conexiones pendientes a la vez.

ct count over N por IP de origen

```
protege la tabla de conexiones
ct original packets ... ct count over N -> drop
```

rate limit

cuántos paquetes por segundo

ct count

cuántas conexiones abiertas a la vez

Son complementarios: uno controla el ritmo, el otro el estado acumulado.

Fuente: Lab 6 v1.0.4 · sección 4, límite de conexiones simultáneas

BONO

Set dinámico con timeout (bono)

Bloquear temporalmente al origen que supera el umbral, y liberarlo solo con el tiempo.

Bloquear temporalmente, no para siempre



1 · SE PASA

supera el umbral

2 · ENTRA AL SET

update @set { ip saddr }

3 · SE DESCARTA

su tráfico se bloquea

4 · EXPIRA

y vuelve a intentar

Con expiración, un falso positivo cuesta unos segundos, no un bloqueo permanente.

Fuente: Lab 6 v1.0.4 · sección 4, flags dynamic,timeout (bono)

Dos controles en el servidor web

SYN cookies y limit_req de nginx: se aplican en el servidor, no en el firewall.

SYN cookies y limit_req: no van en el firewall

SYN cookies

no reserva estado hasta que el cliente responde

limit_req de nginx

peticiones por segundo por cliente

Cada control va en la capa donde el problema se manifiesta.

Fuente: Lab 6 v1.0.4 · sección 4

SÍNTESIS

Lo que queda de la Parte 3

Dos reglas con umbral por origen, tres mitigaciones en nftables y dos controles en el servidor.

Lo que quedó escrito en tus reglas



DETECCIÓN

2 reglas con threshold

track by_src



MITIGACIÓN

rate over · ct count

set con timeout



SERVIDOR

SYN cookies

limit_req

Con esto el check debería dar 6/6 más el bono.

Fuente: Lab 6 v1.0.4 · secciones 3 y 4

SEMANA 13 · PARTE 4

Verificar, reflexionar y cerrar la semana

Parte 4 de 4 · Unidad 3

Ing. Rodolfo Cañas Cervantes

Universidad de la Costa (CUC) · Ingeniería de Sistemas

Semana 13 · Periodo 2026-2 · Barranquilla, Colombia

Los seis objetivos de verificación, las preguntas de reflexión y el puente hacia la IA (inteligencia artificial) de la semana 14.

VERIFICACIÓN

Los seis objetivos y el bono

Línea base, capturas, dos reglas, la mitigación y el cliente que sigue en pie: eso es el check.

Qué verifica ./lab check

1 · línea base

archivo en rango

2 · capturas

cinco campos

3 · regla SYN

al menos 1 alerta

4 · regla HTTP

al menos 1 alerta

5 · nftables

límite de tasa SYN

6 · cliente

siguió con alto % OK

bono · set

dinámico con timeout

El objetivo 6 es el que importa: bloquear sin dañar al usuario legítimo.

Fuente: Lab 6 v1.0.4 · sección 5, Verificación (10 min)

EJECUCIÓN

Cómo se corre el chequeo

./lab ataque syn y luego ./lab check: seis objetivos más el bono, en unos dos minutos.

Atacar y luego verificar

```
en este orden
$ ./lab ataque syn
[OK] ataque syn lanzado
$ ./lab check # 6 objetivos + bono, ~2 min
```

cero alertas

el problema está
en la regla

cliente al 60%

el problema está
en el umbral de mitigación

Fuente: Lab 6 v1.0.4 · sección 5, salida de ./lab check

DIAGNÓSTICO

Si algo falla: cuatro causas típicas

Contenedor caído, regla sin recargar, sid repetido o umbral demasiado estricto.

Solución de problemas del laboratorio

contenedor no arranca

./lab estado y logs del fw

no hay alertas

sid único y ./lab recargar

el ataque llega

falta límite SYN por IP

el cliente se cae

umbral demasiado estricto

Tres de los cuatro se resuelven releyendo lo que escribiste.

Fuente: Lab 6 v1.0.4 · sección Si algo falla

ENTREGA

La entrega: ./lab evidencia

Genera evidencia.tar.gz con la línea base, el análisis, las últimas líneas de eve.json y de nginx.

./lab evidencia genera evidencia.tar.gz



linea-base.txt

lo que mediste antes



analisis-pcap.txt

los cinco campos



últimas líneas

de eve.json



últimas líneas

del log de nginx

Revisa el comprimido antes de subirlo: el análisis suele quedar incompleto.

Fuente: Lab 6 v1.0.4 · sección Entrega

REFLEXIÓN

Las tres preguntas de la reflexión

Por qué no basta el lab, dónde poner los umbrales y cómo ayudaría la IA: se responden por escrito.

Se responden por escrito, no con un comando



1 · ¿por qué no basta?

un DDoS real satura el enlace



2 · ¿dónde el umbral?

sin bloquear al legítimo



3 · ¿cómo ayudaría la IA?

aprender la línea base

Son preguntas abiertas: ahí se ve si el laboratorio dejó algo más que un archivo.

Fuente: Lab 6 v1.0.4 · sección 6, Reflexión (5 min)

El marco legal colombiano

Ley 1273 de 2009: acceder o dañar sistemas ajenos es delito, sin importar la intención.

La ley distingue por autorización, no por intención

ACCESO ABUSIVO

a un sistema ajeno

INTERCEPTACIÓN

de datos

DAÑO INFORMÁTICO

alterar o destruir

SOFTWARE MALICIOSO

uso de programas dañinos

Divulgación responsable: contacta al dueño y respeta su decisión.

Fuente: Ley 1273 de 2009 (Colombia) · delitos informáticos

CONTEXTO

Dónde encaja esto en la Unidad 3

Del ataque a la disponibilidad a la defensa que aprende: el recorrido de la unidad hasta aquí.

El recorrido de la Unidad 3



Cada semana agrega una capa sobre la anterior: hoy escribiste umbrales, la próxima un modelo los aprende.

Fuente: elaboración propia · plan de la Unidad 3

Fuente: elaboración propia; plan de la Unidad 3 del curso

Los límites del laboratorio

Un DDoS volumétrico satura el enlace antes del firewall; eso se resuelve aguas arriba.

Los tres límites del laboratorio



VOLUMÉTRICO

satura el enlace

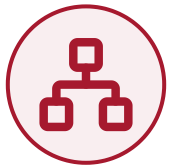
antes del firewall



FALSOS POSITIVOS

umbrales bajos

bloquean legítimos



NO SE ADAPTA

el tráfico cambia

y el umbral envejece

Ese tercer límite es el que abre la puerta a la semana 14.

Fuente: Lab 6 v1.0.4 · sección 4

PUENTE

El puente al Laboratorio 7

Próximamente: un clasificador de tráfico que aprende la línea base en lugar de usar umbrales fijos.

Umbral escrito a mano contra línea base aprendida

HOY · Lab 6

umbral escrito a mano

por origen y ventana



PRÓXIMO · Lab 7

el modelo aprende

la línea base

Las capturas que analizaste hoy te sirven de nuevo: no empezamos de cero.

Fuente: Lab 6 v1.0.4 · puente al Lab 7; PLAN-U3-S14-S16.md

REPASO

Seis ideas que se llevan de la semana

Disponibilidad, tres familias, línea base, umbral por origen, over y verificación del cliente.

Si puedes explicarlas sin mirar, la semana está resuelta



1 · disponibilidad

el objetivo del ataque



2 · tres familias

bps, pps y rps



3 · línea base

primero medir



4 · umbral

por origen y ventana



5 · over

la palabra decisiva



6 · verificar

el cliente sigue en pie

Estas seis son la base sobre la que se construye la semana 14.

Fuente: elaboración propia a partir de Lab 6 v1.0.4

Glosario rápido

SYN flood, HTTP flood, botnet, umbral, limit rate over y evidencia: los términos de la semana.

Vocabulario que ya usaste, no vocabulario nuevo

SYN flood

saludos incompletos

HTTP flood

peticiones en exceso

botnet

equipos que obedecen

umbral

cuenta y ventana

limit rate over

descarta lo que sobra

evidencia

prueba de lo hecho

Fuente: elaboración propia · Lab 6 v1.0.4

Lo que viene: la semana 14

Inteligencia artificial en la defensa y en el ataque: NDR (detección y respuesta en red) con machine learning, deepfakes y SASE (acceso seguro al borde).

Inteligencia artificial en la defensa y en el ataque

DEFENSA

NDR con machine learning

detecta anomalías

ATAQUE

phishing creíble

voces falsas · prompt injection

El umbral que escribiste hoy es el punto de partida de lo que un modelo aprende solo.

Fuente: material de la semana 14; plan de la Unidad 3

REFERENCIAS

Dónde seguir leyendo

La guía del Lab 6, la documentación de Suricata y el wiki de nftables: las tres fuentes de la semana.

Documentación primaria, no blogs

Guía Lab 6 v1.0.4

fuente autoritativa del laboratorio

Documentación Suricata

reglas y opción threshold

Wiki de nftables

rate limit, ct count, sets

Ante una duda de sintaxis, ve primero a la fuente oficial.

Fuente: compilación de fuentes de la semana 13

Cierre de la semana 13

Medir antes de configurar, bloquear sin dañar al legítimo y verificar con evidencia.

Tres frases para llevarse



MEDIR PRIMERO

sin línea base, adivinas



BLOQUEAR SIN DAÑAR

al usuario legítimo



VERIFICAR CON EVIDENCIA

el archivo y el check

La semana 14: la IA entra a la red, y estas tres frases siguen valiendo.

Fuente: elaboración propia · Lab 6 v1.0.4 · Ley 1273 de 2009